

Theory:

1) A high-order function is a function that takes another function as an argument or returns another function.

2) Two examples are

- `map()`

- `filter()`

Other examples are: `reduce()`, and `foreach()`.

3) An anonymous function is a function without a name. It is commonly used as a callback in `setTimeout()`, event listeners, and array methods such as `map()`, `filter()`, and `reduce()`.

4) IIFE stands for immediately Invoked Function Expression. It is a function that executes immediately after being defined. It is used to create a private scope and avoid global variable pollution.

5) Recursion is when a function calls itself to solve a problem. Its two essential parts are.

1) Base case - stops the recursion.

2) Recursive case - calls the function again with a smaller or modified input.

6) It keeps calling itself until the call stack limit is reached causing:

`RangeError: Maximum call stack size exceeded.`

7) A stack overflow occurs when too many recursive function calls fill the Javascript call stack, usually because of infinite or excessively deep recursion.

8) Yes. A function can return another function.

This pattern is commonly used for:

- Closures
- Function factories
- Creating customized functions.

9) * A function that takes a callback receives another function as an argument.

* A function that returns a function gives another function as its result.

10) Recursion is useful for problems with unknown or nested structures such as:

- Nested data
- File folders
- Trees
- Nested arrays.

It naturally solves a problem by repeatedly solving smaller versions of the same problem.

Output prediction:

11) 10

12) LIFE ran

13) 24

14) [2, 4]

15) 3

2

1

done

16) 5

17) 6

18) 20

19) 8

20) hi

Error Finding:

21) There is no base case, so the function calls itself continuously and causes `RangeError: Maximum call stack size exceeded`.

22) An anonymous function cannot be used directly as a function declaration. For an IIFE, it must be treated as a function expression using parentheses.

Correct:

```
(function() {  
    console.log("test");  
})();
```

23) The function is missing a base case.

Correct:

```
if (n <= 1) return 1;
```

without it, recursion never stops.

24) `filter()` returns an array, but the code tries to call that array using `()` as if it were a function.

Correct:

```
const fn = arr.filter(x => x > 2);
```


25) `const fn`: outer stores the reference to outer without calling it. Therefore, `fn()` returns the inner function itself. To first get the returned function:

`const fn = outer();`

Then `fn()` calls inner.

26) Deep recursion can create too many function calls and exceed the call stack limit, causing a stack overflow. For large input a loop or iterative solution is often safer.

27) The function is created but does not execute.

eg) `function() {`

`console.log("Hello");`

`}`

Nothing runs because the final `()` is missing.

28) `return inner();`

calls inner immediately and returns its result.

`return inner;`

returns the function itself. If the goal is to return a function, use the function reference without `()`.

29) The recursive result is not returned to the previous function call, so the final result may become undefined. The recursive call should normally use `return`.

30) Each IIFE creates its own separate private scope. Therefore variables inside one IIFE cannot directly access or share the variables inside another IIFE.

Coding:

```
31) function miniMap (callback, array) {  
    const result = [];  
    for (let i = 0; i < array.length; i++) {  
        result.push (callback (array[i]));  
    }  
    return result;  
}
```

```
const numbers = [1, 2, 3]
```

```
const result = miniMap (function (num) {  
    return num * 2;  
}, numbers);
```

```
console.log (result);
```

Output: [2, 4, 6]

```
32) function factorial (n) {  
    if (n <= 1) {  
        return 1;  
    }  
    return n * factorial (n - 1);  
}
```

```
console.log (factorial (5));
```

Output: 120

```
33) function sumArray (arr) {  
    if (arr.length === 0) {  
        return 0;  
    }  
    return arr[0] + sumArray (arr.slice (1));  
}
```

```
console.log (sumArray ([1, 2, 3, 4]));
```

Output: 10

```

34) const counter = (function() {
    let count = 0;
    return {
        increment: function() {
            count++;
        },
        getValue: function() {
            return count;
        }
    };
})();

```

```

counter.increment();
counter.increment();

```

```

console.log(counter.getValue());    Output: 2

```

```

35) function makeAdder(x) {
    return function(y) {
        return (x+y);
    };
}

```

```

const add5 = makeAdder(5);

```

```

console.log(add5(10));    Output = 15

```

```

36) const cart = [

```

```

    { price: 100, qty: 2 },

```

```

    { price: 50, qty: 3 },

```

```

    { price: 200, qty: 1 },

```

```

];

```

```

const total = cart.reduce(function(acc, item) {

```

```

    return acc + (item.price * item.qty);
}, 0);

```


Console.log (total);

output: 550

37) function reverseString(str) {

if (str.length <= 1) {
return str;

}
return reverseString(str.slice(1)) + str[0];

}
console.log(reverseString("Ahello")); o/p: olleh

38) function flatten(arr) {

if (arr.length === 0) {
return [];

}

const first = arr[0];

const rest = arr.slice(1);

if (Array.isArray(first)) {

return first.concat(flatten(rest));

}

return [first].concat(flatten(rest));

}

console.log(flatten([1, [2, 3], 4, [5, 6]]));

O/P: [1, 2, 3, 4, 5, 6]

39) set Timeout (function() {

console.log("Hello after 2 seconds");

}, 2000);

40) function greet() {

return function() {

return "Hello";

};

}

console.log(greet('')); O/P: Hello.

Real World Scenarios:

- 41) Use `map()` because it transforms every item and returns a new array. A `for` loop also works, but `map()` is more readable and clearly expresses that every item is being transformed.
- 42) Recursion is a natural fit because folders can contain more folders with an unknown depth. The same function can repeatedly process each nested folder until no more folders remain.
- 43) An IIFE creates a private scope. The counter variable remains inside the IIFE and can only be accessed through returned functions such as `increment()` and `get value()`.
- 44) This is a high-order function and closure pattern, also called a function factory. The outer function creates and returns a customized function.
- 45) Very deep recursion creates many function calls and can exceed the Javascript call stack limit. A common alternative is an iterative approach using an explicit stack or queue.
- 46) `reduce()` is designed to combine multiple values into a single result, such as a total. It makes accumulation logic more concise and readable than manually managing

a running total in a for loop.

47) It is a higher-order function pattern because the UI function accepts another function (render) as an argument and uses it to customize behavior or output.

48) An IIFE:

- Executes initialization immediately
- Creates a private scope
- Prevents global variable pollution
- Avoids naming conflicts

49) For a large n , use a loop or iterative approach. Naïve recursion is slow because it repeatedly calculates the same Fibonacci values many times causing unnecessary duplicate calculations.

50) The relies on:

- High-Order functions

- Closures

- Function factory pattern

The outer function returns a customized validation function that remembers the original value such as 5.